

TALLER 1 – INTELIGENCIA ARTIFICIAL

Algoritmos Evolutivos / Algoritmos Genéticos

OBJETIVO GENERAL

Implementar, experimentar y analizar algoritmos genéticos como técnica de Inteligencia Artificial para resolver problemas. El estudiante deberá comparar representación, función de aptitud, selección, cruzamiento, mutación y criterios de parada en diferentes problemas, diseña, explica, y despliega la solución.

PROBLEMA BASE: N-REINAS

El problema original solicita implementar un algoritmo evolutivo para N-Reinas y analizar su rendimiento según las generaciones necesarias para encontrar una solución óptima. Cada solución puede representarse como un vector de tamaño N; el objetivo es minimizar los conflictos hasta llegar a cero.

CONCEPTOS CLAVE

1. ¿Qué es un algoritmo genético y por qué puede considerarse una técnica de búsqueda/optimización dentro de la Inteligencia Artificial?
2. Explique población, individuo, cromosoma, gen, selección, cruzamiento, mutación, elitismo y función de aptitud.
3. ¿Por qué una función de aptitud adecuada es fundamental? Diferencie maximización y minimización.
4. ¿Qué operadores de cruzamiento y mutación son apropiados para cromosomas de permutación y cuáles para cromosomas binarios?
5. ¿Qué riesgos aparecen con una tasa de mutación demasiado baja o demasiado alta?

IMPLEMENTACIÓN – N-REINAS

Implemente el problema original. Cada individuo es un vector de tamaño N donde el índice representa la columna y el valor representa la fila de la reina. La función de aptitud debe cuantificar los conflictos. Pruebe $N=6$ y $N=8$, diferentes tamaños de población y tasas de mutación 0.05, 0.1 y 0.2.

EJERCICIO 1 – PROBLEMA DEL AGENTE VIAJERO (TSP)

Situación: una persona debe visitar N ciudades exactamente una vez y regresar a la ciudad inicial. Se desea encontrar una ruta de costo/distancia mínimo.

Representación: cromosoma = permutación de las ciudades. Ejemplo para 6 ciudades: [0, 3, 1, 5, 2, 4].

Función de aptitud: calcular la distancia total de la ruta, incluyendo el regreso a la ciudad inicial. Para minimizar puede utilizarse una aptitud como $1/(\text{distancia} + \epsilon)$.

Operadores: use OX (Order Crossover), PMX o equivalente; para mutación use intercambio (swap), inversión o inserción.

Flujo:

1. Cree una matriz de distancias para 8, 10 o 15 ciudades.
2. Genere una población inicial de rutas válidas.
3. Implemente selección, cruzamiento, mutación y elitismo.
4. Encuentre la mejor ruta y muestre su distancia.
5. Ejecute al menos 10 veces y calcule mejor, peor y promedio.
6. Compare dos tasas de mutación y explique su efecto.

Preguntas: ¿por qué no puede utilizarse un cruzamiento binario simple sin controlar duplicados? ¿Qué pasa si aumenta el número de ciudades? ¿Qué estrategia de mutación encuentra mejores rutas?

EJERCICIO 2 – ASIGNACIÓN DE CURSOS A SALAS DE CÓMPUTO

Situación: una institución debe asignar cursos a salas y franjas horarias. Se desea minimizar conflictos y penalizaciones.

Datos sugeridos: 8 cursos, 4 salas y 5 franjas. Cada curso tiene número de estudiantes y requisitos como computadores, software o capacidad. Algunas franjas pueden estar bloqueadas.

Representación: cada gen representa la sala y franja asignadas a un curso.

Función de aptitud: penalice sobrecupo, sala sin recursos requeridos, dos cursos en la misma sala/franja y franjas no permitidas. Como objetivo secundario, premie una distribución equilibrada.

Actividad:

1. Diseñe una tabla de cursos con estudiantes y requisitos.
2. Diseñe una tabla de salas con capacidad y recursos.
3. Genere soluciones iniciales y permita soluciones parcialmente inválidas con penalización, o implemente reparación.
4. Minimice la penalización total hasta obtener una programación válida.
5. Presente la solución final como horario.
6. Compare una versión con y sin elitismo.

Preguntas: ¿qué restricciones son duras y cuáles blandas? ¿Es mejor penalizar o reparar individuos inválidos? ¿Cómo cambia el resultado al aumentar el número de cursos?

EJERCICIO 3 – PROBLEMA DE LA MOCHILA (KNAPSACK)

Situación: una mochila tiene capacidad máxima. Hay objetos con peso y valor. Se debe seleccionar una combinación que maximice el valor sin superar la capacidad.

Representación: cromosoma binario. Gen=1 si el objeto se selecciona; gen=0 si no.

Función de aptitud: maximizar la suma de valores. Si el peso supera la capacidad, aplicar penalización o reparación.

Actividad:

1. Utilice al menos 15 objetos con peso y valor diferentes.
2. Pruebe dos capacidades de mochila.
3. Compare penalización frente a reparación.
4. Ejecute varias corridas y registre valor, peso y generación de la mejor solución.
5. Explique por qué este problema permite usar cromosomas binarios.

ENTREGA FINAL

1. Código fuente de los cuatro problemas.
2. Informe con conceptos, metodología, resultados, tablas/gráficas, análisis y conclusiones.
3. Evidencia de experimentación con diferentes parámetros.
4. Conclusión comparativa sobre el uso de algoritmos genéticos como técnica de IA para problemas de búsqueda y optimización.
5. Aplicativo web con el despliegue de estos algoritmos en una interfaz diferentes cada una solucionando los problemas mencionados.